Day 10: Print Numbered Options Using `range()`

1. Day Number

Day 10

2. Topic Name

For loop with `range()`

Today you will learn how to:

- Repeat an action using a `for` loop
- Generate number positions using `range()`
- Access list items using an index
- Print menu options with numbers

---

3. Connection with Yesterday

Yesterday, you repeatedly displayed a menu using a `while` loop.

For example:

```
1. Check Balance
2. Exit
```

But those menu items were probably printed one by one.

Today, you will store all menu options inside a **list** and use a loop to print them automatically.

This is useful because you can add or remove menu options without writing a new `print()` statement for every option.

---

4. Important Topics

`for` loop

A `for` loop repeats a block of code for every value in a sequence.

```
for number in range(3):
    print(number)
```

Output:

```
0
1
2
```

`range()`

`range()` generates a sequence of numbers.

```
range(4)
```

It produces:

```
0, 1, 2, 3
```

The ending number is **not included**.

List

A list stores multiple values in one variable.

```
fruits = ["Apple", "Banana", "Mango"]
```

Indexing idea

Every item in a Python list has a position called an **index**.

Python indexing starts from `0`.

```
Index:    0         1         2
Item:   Apple     Banana     Mango
```

Therefore:

```
fruits[0]
```

gives:

```
Apple
```

---

5. Foundational Notes

Suppose you have this list:

```
options = ["Check Balance", "Deposit Money", "Exit"]
```

The list contains three items.

```
len(options)
```

returns:

```
3
```

Therefore:

```
range(len(options))
```

generates:

```
0, 1, 2
```

These numbers can be used as indexes:

```
options[0] → Check Balance
options[1] → Deposit Money
options[2] → Exit
```

However, users normally expect menu numbering to start from `1`, not `0`.

So the display number can be calculated as:

```
index + 1
```

Important difference:

```
List index:      0, 1, 2
Displayed menu:  1, 2, 3
```

---

6. Easy Example

Consider a list of colors:

```
colors = ["Red", "Green", "Blue"]
```

You can use `range()` to visit each position:

```
for index in range(len(colors)):
    print(index, colors[index])
```

Output:

```
0 Red
1 Green
2 Blue
```

To make the numbering user-friendly, think about displaying `index + 1`.

---

7. Problem Statement

Create a list containing menu options.

Example options:

```
Check Balance
Deposit Money
Withdraw Money
Exit
```

Create a function that prints every option in numbered format using:

- A `for` loop
- `range()`
- List indexing

Expected menu format:

```
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
```

---

8. Concepts Used

Concept	Purpose
List	Stores all menu options
Function	Keeps menu-printing logic separate
`len()`	Finds the number of options
`range()`	Generates list index positions
`for` loop	Repeats the printing process
Indexing	Gets an option from the list
`index + 1`	Creates user-friendly numbering

---

9. Thought Process

Think about the problem step by step:

1. Store all menu options inside one list.
2. Create a function that receives the options list.
3. Find how many options are present using `len()`.
4. Generate indexes using `range()`.
5. Use each index to access the corresponding option.
6. Add `1` to the index for display.
7. Print the number and option together.

For a three-item list:

```
index = 0 → display 1 → first option
index = 1 → display 2 → second option
index = 2 → display 3 → third option
```

---

10. Pseudocode

```
START

CREATE a list of menu options

DEFINE a function that accepts the option list

    FOR every index from 0 to the length of the list
        SET display number to index plus 1
        GET the option using the index
        PRINT display number and option
    END FOR

CALL the function with the menu option list

END
```

---

11. Suggested Solving Approach

Use a **functional approach**.

Suggested structure:

```
Main program
    |
    |-- Create the options list
    |
    |-- Call display_menu(options)
              |
              |-- Loop through indexes
              |-- Print each numbered option
```

Your function could have a responsibility like:

```
display_menu(options)
```

It should only print the menu. It does not need to ask the user for a choice yet.

---

12. Easy Edge Case

Empty option list

Suppose:

```
options = []
```

Then:

```
len(options)
```

is `0`, and:

```
range(0)
```

does not generate any indexes.

Therefore, the loop will not run.

You may decide to display a message such as:

```
No menu options available.
```

A possible thinking rule is:

```
IF the option list is empty
    PRINT an empty-menu message
ELSE
    PRINT the numbered options
```

---

13. Expected Input

This problem does not require keyboard input.

The menu options are stored directly in a list.

Example data:

```
["Check Balance", "Deposit Money", "Withdraw Money", "Exit"]
```

---

14. Expected Output

```
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
```

For an empty list, a suitable output could be:

```
No menu options available.
```

---

15. Hint Only

Use this relationship:

```
range(len(options))
```

The loop variable gives you the list index.

Use:

```
options[index]
```

to access an option, and think about why the displayed number should be:

```
index + 1
```

Try to place this logic inside a function such as `display_menu()`.